

Security

Official security documentation for the XOLVON backend. Decisions behind the controls listed here live in `decision-log.md`; endpoint and auth contracts live in `api-contract.md`.

Security headers

All responses — including CORS preflight replies and error bodies — carry a strict header set produced by [Helmet](#), wired globally in `src/main.ts` before CORS, the validation pipe, and the exception filter. The configuration lives in `src/security/security-headers.ts` and is covered by `src/security/security-headers.spec.ts`.

Header	Value	Rationale
Content-Security-Policy	<code>default-src 'none'; script-src 'self'; frame-ancestors 'none'; connect-src/style-src/img-src/font-src 'self' + configured origins</code>	Deny-by-default policy for any embedded resource. The API serves JSON, so nothing legitimate needs inline scripts or framing; <code>unsafe-inline</code> and <code>unsafe-eval</code> are never emitted (DL-021). <code>frame-ancestors 'none'</code> blocks clickjacking of every response.
Strict-Transport-Security	<code>max-age=31536000; includeSubDomains</code> — only when <code>APP_ENV=production</code>	One-year HTTPS pinning in production. Local and staging intentionally run over plain HTTP, so the header is absent there to avoid locking browsers out of dev origins.
X-Frame-Options	<code>DENY</code>	Legacy framing protection alongside CSP <code>frame-ancestors</code> , for older clients.
X-Content-Type-Options	<code>nosniff</code>	Stops browsers from MIME-sniffing responses into executable/script contexts.
Referrer-Policy	<code>strict-origin-when-cross-origin</code>	Cross-origin requests only receive the origin as referrer, so tokens or identifiers in URLs never leak to third parties; HTTPS→HTTP downgrades send nothing.
X-Powered-By	removed	Express' default fingerprint is disabled at the app level and stripped by Helmet, reducing reconnaissance surface.

CSP origin sources

The fetch/media source lists (`connect-src`, `style-src`, `img-src`, `font-src`) are built at bootstrap from environment variables, always prefixed with `'self'` as the safe default when nothing is configured:

Variable	Feeds	Notes
<code>CSP_CONNECT_SRC</code>	all four source lists	Comma- or space-separated absolute <code>http(s)</code> URLs. Each entry is normalized to its origin; malformed or non- <code>http(s)</code> values are dropped and never echoed into the header.
<code>R2_ENDPOINT</code>	all four source lists	Only the URL origin is used (e.g. <code>https://<account-hash>.r2.cloudflarestorage.com</code>); bucket names, paths, and credentials are excluded.

Approved baseline example (staging/production final origins remain OPEN per DL-021 until the owner confirms domains):

```
CSP_CONNECT_SRC=https://app.xolvon.com,https://admin.xolvon.com
```

No secrets, tokens, or credentials appear in any header value.

Verification

- Unit/config level: `src/security/security-headers.spec.ts` (runs under

`npm run test:esm`) asserts the full header set on a real Nest testing app

for both production (HSTS present) and non-production (HSTS absent), plus origin-parsing edge cases and the no-`unsafe-*` rule.

- Surface level: `curl -s -i` against the local server with and without

`APP_ENV=production` (evidence in the plan notepad, `.omo/.../evidence/`).

Input sanitization

Free-text input is normalized by a bounded, deterministic policy before it can be persisted, implemented in `src/common/sanitization/` and wired per field through the existing global `ValidationPipe` (`whitelist/forbidNonWhitelisted/transform` in `src/main.ts` are unchanged).

Policy — `sanitizeText(value, { maxLength, allowHtml })`:

- Content fields intended for rendered HTML (`allowHtml: false`, the default

and the only mode in use) have every tag-shaped substring (`<...>`) removed (a fixed-point sweep re-checks defensively), so payloads such as

`<scr<script>ipt>` or `<<b>script>` never survive. The backend never

stores raw executable HTML/script content.

- Invisible control characters (NUL, ESC, and the rest of C0/DEL except

`\t \n \r`) are stripped.

- Everything else — including ordinary Unicode (emoji, accented Latin, CJK),

ampersands, and quotes — is preserved byte-for-byte. No trimming, case changes, or truncation: a valid request's visible content is never altered.

- Oversized values are rejected with `400`, not truncated: every sanitized

field carries an explicit `@MaxLength`, and the helper's `maxLength` option throws (never echoes the value) for programmatic callers.

Policy — JSON metadata (`parseSafeJson` / `@IsSafeMetadata`):

- Metadata TEXT fields that may carry structured content (project `tech_stack`)

follow "JSON-shaped means JSON-valid": a value starting with `[` or `{` must parse as a complete JSON document or the request is rejected with `400`;

malformed payloads are never stored for read mappers to silently degrade.

Legacy comma-separated lists remain valid, matching `parseTechStack` in

`src/projects/mappers/project.mapper.ts`.

Sanitized fields (create + update DTOs): `courses.description`,

`lessons.content`, `projects.summary/problem/solution/result`,

`marketplace.description`, `collective.bio`, `orders.notes` (all via

`@SanitizedText`), plus `projects.techStack` via `@IsSafeMetadata`. Plain-text

and identity fields (titles, slugs, names, roles, object keys, tokens) are

deliberately not rewritten — their length/type bounds stay as before.

Known limits: the policy neutralizes markup at the write boundary;

renderers must still apply their own contextual escaping (defense in depth,

on top of the CSP in [Security headers](#)). String items

inside arrays (`skills`, `capabilities`) keep their existing type/size bounds

and are out of scope for tag stripping.

Verification

Xolvon Unit test 2: `src/common/sanitization/sanitization.spec.ts` (runs under

`npm run test:esm`) — 27 tests covering tag removal, reassembly payloads,

Unicode preservation, control characters, length bounds, idempotence,

malformed/valid JSON metadata, and the real content DTOs.

- Surface level: the T7 block in `test/app.e2e-spec.ts` asserts that markup is

gone from the actual DB `INSERT` parameters, oversized content returns 400

before any write, and malformed `techStack` JSON returns 400; manual

`curl` evidence lives in the plan notepad,

`.omo/notepads/xolvon-addendum-hardening-docs/evidence/`.

Consolidated security controls

Every control introduced or hardened by the addendum plan

(`.omo/plans/xolvon-addendum-hardening-docs.md`), mapped to its code and to

the addendum §5 checklist (`.omo/drafts/xolvon-addendum-hardening-docs.md`,

Scope ledger — In scope). All entries verified against the source tree on

2026-09-12.

Control (plan task)	Code	Tests	Addendum §5 checklist item	Decision
Request integrity / CSRF posture (T3)	<code>src/common/guards/request-integrity.guard.ts</code> — global <code>APP_GUARD</code> (<code>src/app.module.ts:66</code>); <code>POST/PATCH/DELETE</code> only; rejects <code>Sec-Fetch-Site: cross-site</code> and any <code>Origin / Referer</code> outside the allowed set; <code>GET/HEAD/OPTIONS</code> and headerless non-browser requests pass	<code>request-integrity.guard.spec.ts</code> + <code>e2e</code> + <code>curl</code> matrix	"Bearer-only request-integrity/Origin/Fetch-Metadata guard for mutation routes and negative tests" — no cookie-session CSRF tokens	DL-018 (with DL-003)
Security headers (T4)	<code>src/security/security-headers.ts</code> wired at <code>src/main.ts:35–39</code> , before <code>CORS/pipes/filter</code> — see Security headers	<code>src/security/security-headers.spec.ts</code> + <code>curl</code> headers <code>local/ APP_ENV=production</code>	"Helmet security headers with production-only HSTS and explicit CSP/frame/nosniff/referrer policy"	DL-021
CORS whitelist (T5)	<code>src/config/cors.ts</code> (<code>resolveCorsOrigins</code>) + <code>app.enableCors({ origin, credentials: true })</code> at <code>src/main.ts:43–50</code> ; wildcards rejected with credentials; local fallback <code>http://localhost:3001</code>	<code>cors.spec.ts</code> + <code>curl</code> allowed/denied origins	"Explicit CORS origin whitelist per environment"	DL-020
Route-specific throttling (T6)	<code>src/config/throttle.config.ts</code> (5 named groups: default 100, search 30, catalog 30, signed 10, order 10 / min) + <code>@Throttle</code> overrides on controllers; auth login/register 5/min (<code>src/auth/auth.controller.ts:33,42</code>); health skips all groups (<code>src/app.controller.ts:10</code>)	<code>throttle.config.spec.ts</code> + <code>e2e</code> burst tests per group	"Route-specific throttling for public search/catalogs, signed URL, and order creation"	DL-012, DL-016
Trust proxy / client-IP (T6)	<code>src/config/trust-proxy.ts</code> + <code>app.set('trust proxy', ...)</code> (<code>src/main.ts:25–28</code>): unset → <code>false</code> (fail closed against <code>X-Forwarded-For</code> spoofing of throttle buckets), <code>1</code> for one proxy hop, malformed → bootstrap aborts	<code>trust-proxy.spec.ts</code> + live burst with rotating XFF	Throttling correctness prerequisite of the checklist item above	DL-016 (extension)
Input sanitization (T7)	<code>src/common/sanitization/</code> (<code>sanitizeText</code> , <code>parseSafeJson</code> , <code>@SanitizedText</code> , <code>@IsSafeMetadata</code>) wired through the existing global <code>ValidationPipe</code> — see Input sanitization	<code>sanitization.spec.ts</code> (27 tests) + <code>e2e</code> DB read-back	"Input/content sanitization policy for HTML-bearing strings and length limits, with tests and no silent data corruption"	RULES §95, DL-023 lineage
Password hashing	Argon2id via <code>PasswordService</code> (<code>src/auth/</code>); never exposed in responses	auth unit/e2e	Baseline (pre-addendum), kept	DL-003 context
Refresh-session hygiene	SHA-256 hashed <code>sessions.refresh_token</code> , IP/user-agent capture, expiry purge on refresh, delete on logout (<code>src/auth/auth.service.ts:136–225</code>)	auth unit/e2e	Baseline, kept	DL-003

Control (plan task)	Code	Tests	Addendum §5 checklist item	Decision
Input contract enforcement	Global ValidationPipe whitelist + forbidNonWhitelisted + transform (src/main.ts:53–59); DTOs reject unknown properties	every controller spec relies on it	Baseline, kept	—
Safe error surface	AllExceptionsFilter (src/common/filters/all-exceptions.filter.ts): generic 500, no stack/upstream-body/secrets	filter + e2e assertions	Baseline, kept	—
Immutable admin audit	AuditService.record() → admin_audit_logs for verify/activate/cancel/revoke/publish/create/update/delete; no public endpoint, metadata never leaves the API	audit unit + admin e2e	Baseline, kept (HANDBOOK §7 obligation)	—

Dependency audit status

`npm audit --json` was captured during plan T2 on **2026-09-12**

(evidence: `.omo/notepads/xolvon-addendum-hardening-docs/evidence/t2-audit.json`,

exit code 1 as expected with nonzero findings):

Severity	Count
Critical	0
High	4
Moderate	1
Low	2
Total	7

Remediation of the high/moderate residuals and the Dependabot-vs-scanner policy are plan T13 work; no paid scanner has been adopted (DL-023 — OPEN).

No claim of a clean audit is made here.

Known limitations (honest state)

- 1. Production D1/R2 live path is BLOCKED, not deployed.** There is no verified staging/production deployment target, secret store, or HTTPS domain in this workspace, and no live evidence is fabricated — see `qa-results.md`. All security verification above is local (unit/e2e/curl against locally booted instances with non-secret config).
- 2. CSP and CORS final origins are OPEN DECISIONS.** Staging/production frontend origins and `CSP_CONNECT_SRC` values await owner confirmation (DL-020, DL-021, DL-027); until then the lists default to `'self'` plus the configured `R2_ENDPOINT` origin only.
- 3. Monitoring/alerting is interface-only.** `MonitoringPort` (`src/observability/monitoring.port.ts`) has no implementation or provider; vendor and recipients remain OPEN (DL-022).
- 4. **The request-integrity guard is a browser-CSRF control, not an auth layer.**** A `POST/PATCH/DELETE` with no `Origin`, no `Referer`, and no `Sec-Fetch-Site` header (curl, Postman, server-to-service clients) passes the guard by design; such clients still need valid Bearer credentials on every protected route. This is the explicit non-browser bypass policy recorded under DL-018, not an oversight.
- 5. Sanitization neutralizes markup at the write boundary only;** renderers

must still apply contextual escaping (defense in depth over CSP).